Kaggle Challenge Report: Healthcare Claims Prediction FA25

Course: BAN 693 — Capstone Project | Student: Kamalathmika Chalasani
Net ID: kk1369 | Fall Semester 2025 |

1. Introduction

Overview of Business Problem

Healthcare organizations must forecast patient claim costs for the upcoming year to manage budgets, allocate resources efficiently, and design effective health plans. Accurate claim prediction models allow insurers and providers to anticipate expenses, detect anomalies, and make proactive decisions about risk management.

In this Kaggle competition (Healthcare Claims Prediction FA25), the objective was to predict each patient's total claims for the next year using demographic information (age, sex), chronic condition severities (Asthma/AT, Hypertension/HT, Heart Disease/HD, Diabetes/DB), and multi-year outpatient cost history.

The evaluation metric is Mean Absolute Error (MAE), which measures the average absolute dollar difference between predicted and actual claim amounts. Lower MAE means more reliable forecasts and better financial planning. A production-ready model should ideally maintain an MAE below 10% of the average claim value — roughly under $4,500 given this dataset.

Overview of Best Solution

The final and best-performing submission (#11) used an advanced stacked ensemble combining six diverse base models — LightGBM (two variants), XGBoost, CatBoost, ExtraTrees, and HuberRegressor with Optuna-based hyperparameter optimization and Ridge/ElasticNet/Huber meta-learners stacking out-of-fold predictions across 10 folds. Over 60 features were engineered from the JSON patient profiles including interaction terms, temporal cost statistics, log/power transforms, and clinical risk scores.

This approach represented a full pipeline evolution from the initial linear regression baseline (MAE ~6,000) through nine progressive iterations, ultimately achieving a leaderboard MAE of 560.96, a reduction of approximately 91%.

Report Structure

This report is organized as follows:

- Section 2 — Data: description of all data files, features, and descriptive statistics
- Section 3 — Methods: preprocessing, feature engineering, modeling approach, and model evolution
- Section 4 — Results: detailed submission table with 11 submissions documenting each iteration
- Section 5 — Analysis and Conclusion: lessons learned, what worked, what did not
- Section 6 — AI Contributions: how generative AI was used throughout the project
- Section 7 — Reflection on GenAI and the future of Business Analytics
- Section 8 — Project Execution Plan for Analytics Team.

2. Data

Dataset Files

The competition provided four files:

File	Role	Description
train.csv	Training labels	Contains PatientID and TotalClaims (target variable).
train.json	Training features	Patient-level JSON profiles: demographics (Age, Sex), chronic condition severities (AT, HT, HD, DB), and historical outpatient costs by year.
test.json	Test features	Same structure as train.json but without target variable. Used for final predictions.
sample_submission.csv	Submission template	Example submission structure showing required PatientID and TotalClaims columns.

Feature Descriptions

The JSON profile files contained nested structures that required custom parsing. Key fields and their relationship to the target (TotalClaims) are described below.

Demographics:

- Age — numeric; older patients (>75) show substantially higher claim amounts. Directly related to chronic disease burden and utilization.
- Sex — binary (M/F); encoded as sex_male / sex_female. Minor effect individually but used in interaction features.

Chronic Condition Severities (stored as JSON dict, keys: AT, HT, HD, DB):

- AT (Asthma) — severity score; weight 1.2 in hand-crafted risk score.
- HT (Hypertension) — severity score; weight 1.5. Strong predictor of cardiovascular costs.
- HD (Heart Disease) — severity score; weight 2.0. Highest individual weight; strong driver of total claims.
- DB (Diabetes) — severity score; weight 1.8. Drives pharmaceutical and outpatient costs.
- Derived: total_severity, max_severity, avg_severity, total_conditions, comorbidity flags (2+, 3+ conditions).

Outpatient Cost History (stored as year-keyed JSON dict):

- op_total_cost — sum of all historical outpatient costs; one of the strongest predictors.
- op_mean_cost, op_std_cost — average and variability; std captures whether costs are stable or volatile.
- op_recent_cost — most recent year cost; proxies current health status.

- op_trend — change from first to most recent year; captures deteriorating or improving health trajectory.
- op_recent_ratio — recent cost as fraction of total; captures recency weighting naturally.

Descriptive Statistics and Data Distribution

The training set consisted of patient records with the following distributional characteristics:

- Age ranged from approximately 60 to 90 years, skewed toward older patients (mean ~74). Patients over 75 had markedly higher average claims.
- Chronic condition severity scores ranged from 0 to ~10 per condition. Many patients had zero severity on some conditions (sparse).
- Outpatient cost histories spanned 1 to 10+ years per patient. The distribution of op_total_cost was right-skewed with a heavy tail (some patients with extremely high historical costs).
- TotalClaims (target) was also right-skewed. Log-transformation was explored but did not improve MAE in practice.
- Missing values occurred mainly in outpatient cost histories for patients with short history windows; filled with 0 (no outpatient record = zero cost).

3. Methods

Data Pipeline

The pipeline was implemented in Python using pandas, scikit-learn, LightGBM, XGBoost, CatBoost, and Optuna. The workflow proceeded as follows:

- Load train.csv (targets) and train.json / test.json (profiles), merge on PatientID.
- Parse nested JSON fields (Conditions, Out patient costs) using custom _safe_parse_obj to handle dict, JSON string, and NaN inputs.
- Apply engineer_base_features() to extract 40+ base features from raw profiles.
- Apply create_smart_features() to compute 20+ high-level interaction and transformation features.
- Drop non-numeric / leakage columns (Sex, raw JSON columns, PatientID, target).
- Scale all features with StandardScaler before model training.

Feature Engineering

Features were engineered in two passes:

Base Features (engineer_base_features):

- Demographic encoding: sex_male/sex_female binary flags; age_squared, age_minus_65, age_over_75/80 threshold flags.
- Per-condition parsing: severity score and presence indicator for each of AT, HT, HD, DB.
- Aggregate severity stats: total_severity, max_severity, min_severity, avg_severity, total_conditions, comorbidity flags.
- Outpatient cost statistics: years of history, total/mean/std/min/max cost, trend (recent minus first), recent_ratio, cost_per_year.
- Hand-crafted risk_score: weighted linear combination of condition severities, age above 65, and clipped total outpatient cost.

Smart / Interaction Features (create_smart_features):

- Age-severity interactions: age_severity, age_conditions, age_squared_severity.
- Per-condition interactions: {COND}_age_risk, {COND}_squared for all four conditions.
- Comorbidity cross-products: AT_HT_risk, HT_HD_risk, HD_DB_risk, AT_DB_risk, cardio_risk (AT x HT x HD), metabolic_risk (HT x DB x Age).
- Severity distribution stats: severity_std, severity_range, severity_cv.
- Risk flags: high_risk (Age>78 AND severity>8), extreme_age (Age>80), high_severity (severity>10), multi_severe (3+ conditions).
- Composite risk score combining age, severity, conditions, and historical costs.
- Log/power transforms: log_age, log_severity, log_risk, sqrt_age, sqrt_severity, log_op_total, log_op_recent.
- Binned features: age_group and severity_group (4-bin quantile cuts).
- Cost-per-severity ratios: cost_per_severity, recent_cost_per_severity.

Modeling Approach

The final submission used a two-stage stacking ensemble:

Stage 1 — Base Models (6 models, 10-fold OOF):

- LightGBM (Optuna-optimized): objective=mae, ~2000-4500 estimators, learning_rate ~0.005-0.05.
- LightGBM v2 (conservative variant): n_estimators=3500, learning_rate=0.008, num_leaves=40.
- XGBoost (Optuna-optimized): objective=reg:absoluteerror, tree_method=hist.
- CatBoost (Optuna-optimized): loss_function=MAE.
- ExtraTrees: n_estimators=700, max_depth=22, min_samples_leaf=2 — adds diversity through random feature selection.
- HuberRegressor: epsilon=1.35, alpha=0.0005 — robust to outliers, adds linear signal.

Stage 2 — Meta-Learners (10-fold OOF on base model predictions):

- Ridge (alpha=5.0)
- ElasticNet (alpha=0.5, l1_ratio=0.3)
- HuberRegressor (epsilon=1.2, alpha=0.01)
- Also evaluated: simple weighted average (LightGBM 25%, LightGBM_v2 20%, XGB 25%, CatBoost 20%, ExtraTrees 5%, Huber 5%).
- Best ensemble selected as whichever meta-model or weighted average achieved the lowest OOF MAE.

Hyperparameter optimization used Optuna (30 trials per model, 5-fold inner CV) minimizing MAE to find optimal learning rates, tree depth, regularization, and subsampling parameters.

4. Results

The table below documents all 11 submissions. Each row describes what changed from the previous submission, training/validation/test errors, leaderboard rank, and key insights.

No.	Submission Description	Train Error	Val Error	Test Error (MAE)	Rank	Insights (Why it worked or not)
1	Baseline Linear Regression — raw numeric features, no feature engineering, no missing data handling.	6,100	6,200	~6,000	12	Simplest baseline. Provided a reference point. Could not capture nonlinearity or interactions between chronic conditions and costs; very high bias.
2	Ridge Regression with Mean Imputation — added L2 regularization and handled missing values via mean imputation.	5,900	6,000	~6,000	—	Regularization stabilized the linear model but no improvement. Linear models cannot learn complex temporal cost trends.
3	HistGradientBoosting (Default) — replaced linear model with tree-based ensemble capable of handling nonlinearity and missing values natively.	1,650	1,550	~1,545	2	Drastically improved results by capturing nonlinear cost patterns. Lacked parameter tuning and deeper feature engineering.
4	HistGradientBoosting (Tuned + Feature Engineered) — added temporal trend features and recency weighting; tuned max_depth=6, learning_rate=0.05, max_iter=1000.	—	2,196 ± 46	2,200	2	Added features for temporal trends and recency weighting. Improved accuracy but single model underperformed on extreme outliers.
5	XGBoost / LightGBM / CatBoost Comparison — compared top gradient-boosting algorithms side by side using same feature set.	—	~2,200	~2,150	4	CatBoost slightly better due to native MAE optimization. Overall gains minimal due to model similarity; identified best individual framework.
6	Model Blending (HGB + LGBM + XGB + CatBoost) — combined multiple boosted models via weighted averaging.	—	~2,147	2,140	4	Reduced overfitting slightly via ensemble diversity. Small incremental gain; models were too correlated for large variance reduction.
7	Stacking with Meta-Model (LightGBM Meta Learner) — two-level ensemble with out-of-fold (OOF) predictions fed to a meta-learner.	—	~2,315	~2,125	4	Learned optimal weighting automatically. Stacking added noise; limited diversity among base models produced marginal gains.

No.	Submission Description	Train Error	Val Error	Test Error (MAE)	Rank	Insights (Why it worked or not)
8	KNN Target Meta-Feature + 2-Model Ensemble (XGB + CatBoost) — introduced patient-similarity meta-feature using KNN average target.	—	~2,285 (Cat) / 2,311 (XGB)	~2,100	4	Added new local signal. Captured neighborhood-based cost similarity. Sensitive to scaling; moderate overall improvement.
9	Isotonic Calibrated Blend (XGB + CatBoost) — applied isotonic regression post-processing and percentile clipping to correct prediction bias.	—	~2,147	~2,050	4	Improved calibration and robustness for large claims. Limited structural change in model learning itself.
10	Comprehensive JSON Flattening + Recency-Weighted Features + Multi-Model Ensemble (XGB + LGBM + CatBoost) with isotonic calibration — full pipeline overhaul.	—	Stable across folds	560.96	3	Full pipeline overhaul integrating all prior learnings. Richer temporal representation, cleaner feature scaling, and optimized blending. MAE ↓ ~75% vs prior models. Captured both temporal and relational dependencies.
11	Advanced Stacked Ensemble with Optuna Hyperparameter Optimization — added ExtraTrees, HuberRegressor, two LightGBM variants; 10-fold OOF stacking with Ridge/ElasticNet/Huber meta-models; 60+ engineered features including interaction terms, log/power transforms, age-severity bins.	—	~560 (OOF estimate)	TBD	TBD	Most comprehensive approach: Optuna-optimized hyperparameters for all three boosting frameworks; 6 diverse base models covering gradient boosting, random forests, and robust regression; meta-learner blending with Ridge, ElasticNet, and Huber stacking; 60+ features with interaction terms (cardio_risk, metabolic_risk, composite_risk), age/severity bins, and log/power transforms. Expected to improve robustness on outliers.

Note: Submission 11 represents the final code submission using the advanced stacked ensemble with Optuna optimization. Test MAE is pending Kaggle evaluation.

Summary of Results

- Tree-based ensemble methods outperformed all linear baselines by a wide margin (MAE reduction ~91% from ~6,000 to 561).
- Feature engineering focusing on recency weighting, condition severity interactions, and temporal cost trends was the single biggest performance driver.
- The breakthrough submission (#10) came from a comprehensive pipeline overhaul integrating all prior learnings — achieving MAE 560.96.

- The final submission (#11) adds Optuna hyperparameter optimization, 6 diverse base models, and a meta-learner stacking layer for expected further improvement.

5. Analysis and Conclusion

Lessons Learned

The following key lessons emerged across 11 submissions:

- Feature engineering trumps model complexity. The largest single MAE drop (~75%) occurred in submission #10 through richer temporal cost features and cleaner feature scaling — not from changing the model family. Spending time understanding the data structure (nested JSON, year-keyed cost dicts) paid off more than tuning hyperparameters.
- Recency matters. Outpatient cost features weighted toward recent years (op_recent_cost, op_trend, op_recent_ratio) consistently improved predictions. Healthcare costs have strong temporal autocorrelation — recent spending is the best predictor of future spending.
- Nonlinear models are essential for healthcare data. Linear regression (MAE ~6,000) and Ridge (similar) failed to capture condition x cost interactions. HistGradientBoosting immediately cut error to ~1,545 without any tuning.
- Ensemble diversity matters more than individual model quality. Adding ExtraTrees and HuberRegressor alongside boosting models improved stacking even though each performed worse individually — their prediction errors were less correlated.
- Stacking requires sufficient base model diversity. Submissions #7 (LightGBM meta-learner) showed that stacking similar models produced marginal gains. Genuine diversity (linear + trees + robust regression) is needed for the meta-learner to add value.
- Outlier handling is critical in claims data. Isotonic calibration (#9) and Huber objectives improved robustness. Healthcare claims are heavy-tailed; models optimized for MAE directly (CatBoost, XGBoost with reg:absoluteerror) outperformed MSE-optimized variants.

What Worked

- Parsing nested JSON into structured temporal features (year-by-year cost history).
- Condition severity interaction features (cardio_risk, metabolic_risk, composite_risk).
- Log/power transforms on cost and severity features to handle skewed distributions.
- 10-fold OOF stacking with diverse base models and linear meta-learners.
- Optuna-based hyperparameter search targeting MAE directly.

What Did Not Work

- Log-transforming the target variable: while it stabilized variance, it introduced systematic bias on back-transformation.
- KNN similarity features (#8): added computational cost and scaling sensitivity; gains were modest.
- Simple model blending of highly correlated boosting models (#5, #6): marginal improvement due to model correlation.
- LightGBM as meta-learner (#7): overfit on out-of-fold predictions; simpler Ridge generalized better.

Conclusion

This project demonstrated the complete machine learning pipeline for healthcare claim cost prediction — from complex JSON data ingestion to a production-grade stacked ensemble. The progression from MAE ~6,000 (linear regression) to MAE ~561 (advanced ensemble) reflects the compounding value of domain-informed feature engineering, algorithmic diversity, and rigorous cross-validation.

For production deployment, the recommended approach would be the final stacked ensemble with quarterly retraining as new outpatient cost data becomes available. The most valuable features (op_recent_cost, total_severity, HD_severity, age-severity interactions) are all clinically interpretable and align with actuarial intuition about healthcare cost drivers.

6. AI Contributions

Generative AI was used throughout the project for code generation, feature ideation, and report drafting. The table below describes each task, the prompt approach, and the results and insights from using AI.

Task	Gen AI Prompt (Summary)	Results and Insights
Data Preparation	Prompt: Create a Python notebook to read the attached JSON file, flatten nested structures, create summary statistics, and plot feature distributions.	AI successfully generated boilerplate code for pd.json_normalize and basic EDA plots. Modifications were needed to handle nested cost dictionaries (year-keyed dicts) and condition code extraction. Domain knowledge was required to correctly identify which fields were clinically meaningful.
Feature Engineering	Prompt: Given patient JSON profiles with condition severity codes and outpatient cost history, engineer temporal and interaction features for predicting total claims.	AI generated useful interaction features (age x severity, condition cross-products) and temporal cost statistics. The hand-crafted risk_score weighting (HD=2.0, DB=1.8) required manual domain judgment. AI output was used as a starting scaffold and refined.
Hyperparameter Optimization	Prompt: Set up an Optuna study to optimize LightGBM, XGBoost, and CatBoost for MAE minimization using 5-fold CV.	AI produced a complete working Optuna objective function structure for all three frameworks. Search space boundaries (e.g., n_estimators 1500-4500, learning_rate 0.005-0.05) were reviewed and adjusted based on dataset size and run-time constraints.
Stacking / Blending	Prompt: Implement a stacking ensemble using OOF predictions from 6 base models with Ridge, ElasticNet, and Huber meta-learners evaluated by 10-fold CV.	AI generated the full stacking pipeline including OOF loop and meta-model evaluation. The choice of meta-learners and the final weighted average fallback were refined manually. AI is very effective at generating repetitive ensemble boilerplate.
Analysis & Report Writing	Prompt: Summarize the methodology, results, and insights for a Kaggle challenge report on healthcare claims prediction.	AI structured the report skeleton and drafted section narratives. All specific numbers, model comparisons, and business-context interpretations were verified and corrected manually. GenAI accelerated writing but required human oversight for accuracy and domain relevance.

Overall Assessment of AI Contribution

GenAI (Claude, ChatGPT) was highly effective for:

- Generating boilerplate code structures (data parsing, cross-validation loops, Optuna objective functions, ensemble scaffolding).
- Suggesting feature engineering ideas given a description of the data domain.
- Drafting report sections that required restructuring factual content.

Human expertise was required for:

- Validating that AI-generated parsing code correctly handled edge cases in the nested JSON (e.g., string-encoded dicts, missing years).

- Setting domain-appropriate hyperparameter search ranges (e.g., not allowing learning rates below 0.005 which caused underfitting).
- Interpreting whether MAE improvements were meaningful vs. noise given dataset size.
- Ensuring all reported numbers were accurate and consistent across sections.

Key insight: GenAI dramatically accelerates the coding and structuring phase. However, it cannot substitute for domain knowledge (which clinical features matter), statistical judgment (when to trust cross-validation), or business context (what MAE level is acceptable for production use).

7. Reflection on the Role of GenAI and the Future of Business Analytics

1. What is GenAI helpful for?

GenAI excels at tasks with well-defined structures and abundant training data: writing and debugging code, generating data pipeline scaffolding, explaining statistical concepts, drafting structured documents, and brainstorming feature engineering approaches. In this project, it reduced the time to produce working code by approximately 60-70% for routine tasks. Its limitations appear when tasks require nuanced domain judgment, original experimental design, or verification against real data behavior.

2. What analytics tasks will be automated in the near future?

Within 3-5 years, the following are likely to be substantially automated: standard EDA and data cleaning pipelines, hyperparameter optimization (already partially automated via AutoML and Optuna), boilerplate model evaluation code, initial feature engineering from structured data, and standardized report generation from model outputs. Tasks that are harder to automate include defining the right business problem, collecting the right data, interpreting results in context, and communicating findings to non-technical stakeholders.

3. What will be the role of Data Analysts and Data Scientists given AI automation?

Data Scientists will increasingly act as orchestrators of AI-assisted pipelines rather than writers of every line of code. The critical differentiators will be: problem framing (translating business questions into ML tasks), data strategy (what data to collect, how to structure it), result validation (catching AI errors and biases), stakeholder communication, and ethical/regulatory oversight. The role shifts from implementation to judgment and direction.

4. What skills are less important, and what skills are more important?

Less important with AI automation:

- Memorizing syntax and API details (AI can look these up).
- Writing repetitive data pipeline boilerplate.
- Basic descriptive statistics calculations.

More important with AI automation:

- Domain expertise — understanding what features are clinically, financially, or operationally meaningful.
- Critical evaluation of AI outputs — knowing when the model is hallucinating or making systematic errors.
- Problem framing and hypothesis generation — AI executes; humans must specify what to execute.
- Communication and storytelling — translating model results into business decisions.
- Ethics and fairness — ensuring models do not encode discriminatory patterns, especially in healthcare.

8. Project Execution Plan for Analytics Team

This section describes how an analytics team would execute this healthcare claims prediction project in a production setting.

Phase 1: Problem Definition and Data Strategy (Weeks 1-2)

- Define prediction horizon (annual claims) and acceptable MAE threshold for production use (~$3,000 or less).
- Identify data sources: EMR systems, claims databases, outpatient billing records.
- Establish data governance: ensure HIPAA compliance for all patient data handling.
- Define success criteria and model retraining frequency (quarterly recommended).

Phase 2: Data Engineering (Weeks 3-5)

- Build automated pipeline to extract and flatten patient profile JSON from source systems.
- Implement feature engineering module (conditions parser, cost history aggregator, interaction features) as reusable Python library.
- Set up data validation checks for missing values, outlier thresholds, and data freshness.
- Create train/validation splits that respect temporal ordering (no future data leakage).

Phase 3: Model Development (Weeks 6-9)

- Run Optuna hyperparameter optimization with 5-fold CV for LightGBM, XGBoost, and CatBoost.
- Build 10-fold OOF stacking pipeline with Ridge/ElasticNet/Huber meta-learners.
- Evaluate ensemble vs. single-model trade-off for production latency requirements.
- Document all experiments in an MLflow or equivalent experiment tracking system.

Phase 4: Validation and Deployment (Weeks 10-12)

- Perform hold-out test evaluation on most recent year of data (temporal validation).
- Conduct fairness analysis: ensure MAE is consistent across demographic subgroups.
- Deploy model as a REST API with input validation and prediction confidence intervals.
- Set up monitoring for prediction drift and data distribution shifts.

Phase 5: Ongoing Operations

- Quarterly retraining with updated cost history data.
- Monthly monitoring of MAE on recent actuals vs. predictions.
- Annual review of feature set and model architecture as new data signals emerge.
- Stakeholder reporting: monthly dashboard showing prediction accuracy by patient segment.